Глубокое обучение с подкреплением

Модуль 7: Многоагентное обучение с подкреплением

Курс Hugging Face Deep RL

Полная учебная версия на основе MDX

Источник: курс Deep RL от Hugging Face (оригинальные материалы на английском языке).

Содержание

1 Введение **2**
- 1.1 Уведомление о поддержке курса . . . 3

2 Введение в многоагентное обучение с подкреплением (MARL) **3**
- 2.1 От одного агента к нескольким . . . 3
- 2.2 Типы многоагентных сред . . . 5

3 Проектирование многоагентных систем **6**
- 3.1 Децентрализованная система . . . 6
- 3.2 Централизованный подход . . . 7

4 Self-Play: классический метод обучения агентов в состязательных играх **7**
- 4.1 Что такое Self-Play? . . . 8
- 4.2 Self-Play в ML-Agents . . . 9
- 4.3 Рейтинг Elo для оценки агента . . . 9

5 Практическое задание **12**
- 5.1 Что такое AI vs. AI? . . . 12
- 5.2 Шаг 0: установка ML-Agents и загрузка среды . . . 13
- 5.3 Шаг 1: знакомство со средой . . . 14
- 5.4 Шаг 2: знакомство с MA-POCA . . . 15
- 5.5 Шаг 3: настройка файла конфигурации . . . 16
- 5.6 Шаг 4: запуск обучения . . . 17
- 5.7 Шаг 5: публикация агента на Hugging Face Hub . . . 17
- 5.8 Шаг 6: проверка готовности модели к AI vs. AI . . . 19
- 5.9 Шаг 7: просмотр матчей в демонстрации . . . 20

6 Проверка знаний **20**
- 6.1 Q1: выберите правильное соответствие типов многоагентных сред . . . 20
- 6.2 Q2: какие утверждения верны для децентрализованного обучения? . . . 20
- 6.3 Q3: какие утверждения верны для централизованного обучения? . . . 20
- 6.4 Q4: своими словами объясните подход Self-Play . . . 20
- 6.5 Q5: сопоставьте описания с гиперпараметрами Self-Play . . . 20
- 6.6 Q6: каковы основные причины использования рейтинга Elo? . . . 21
- 6.7 Ответы и пояснения к тесту . . . 21

7 Заключение **22**
- 7.1 Продолжайте учиться и оставайтесь молодцами . . . 22

8 Дополнительная литература **22**
- 8.1 Введение в многоагентные системы . . . 22
- 8.2 Self-Play и MA-POCA . . . 22

1 Введение



Рис. 1: Обложка курса глубокого обучения с подкреплением Hugging Face [оригинал]

С самого начала курса мы учились обучать агентов в *одноагентной системе*, где агент находился в среде один: он **не сотрудничал и не взаимодействовал с другими агентами**.

Этот подход хорошо работал, а одноагентные системы полезны во множестве приложений.



Рис. 2: Коллаж сред, в которых вы обучали агентов с начала курса [оригинал]

Но мы, люди, **живём в многоагентном мире**. Наш интеллект развивается во взаимодействии с другими участниками. Поэтому наша **цель — создавать агентов, способных взаимодействовать с людьми и другими агентами**.

Следовательно, нам нужно изучить обучение агентов с помощью глубокого обучения с подкреплением в *многоагентной системе*, чтобы строить устойчивых агентов, способных адаптироваться, сотрудничать и конкурировать.

Сегодня мы **познакомимся с основами увлекательной области многоагентного обучения с подкреплением (MARL)**.

Самое интересное: в этом модуле вы впервые обучите агентов в многоагентной системе — **футбольную команду из двух агентов, которая должна победить команду соперников в матче 2 на 2**.

1.1 Уведомление о поддержке курса

Обратите внимание: **курс Deep Reinforcement Learning теперь поддерживается в ограниченном режиме**. Тем не менее он **по-прежнему остаётся прекрасным ресурсом для изучения теории и практики глубокого обучения с подкреплением**.

Необходимо учитывать следующее:

- *Модуль 7 (AI vs AI)*: сейчас этот функционал не работает. Однако вы по-прежнему можете обучить агента играть в футбол и наблюдать за его поведением. Таблица лидеров соревнования AI vs AI по футболу была отключена.

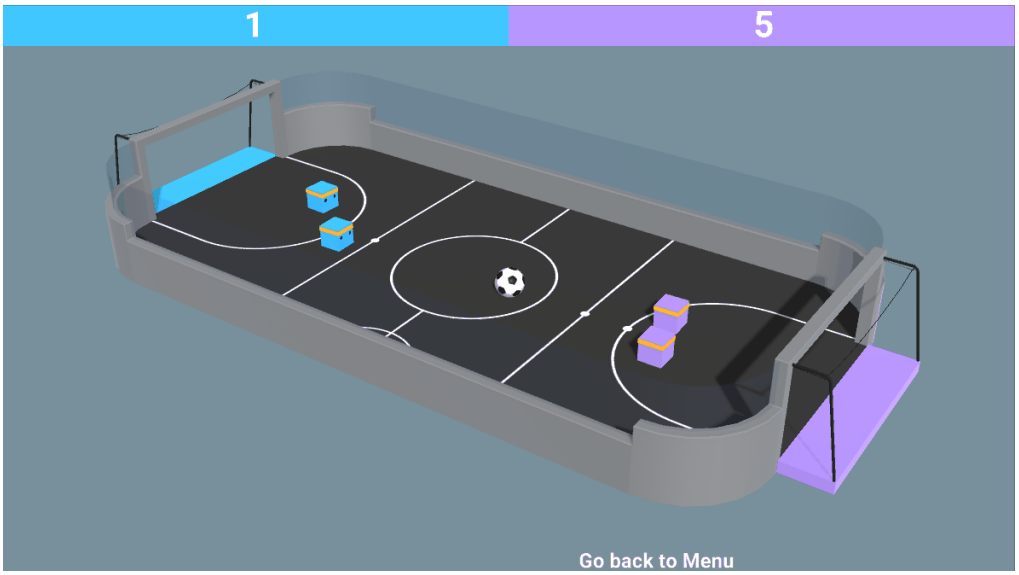


Рис. 3: SoccerTwos: футбольная среда Unity ML-Agents для игры два на два (команда Unity ML-Agents) [оригинальная анимация]

Итак, приступим!

2 Введение в многоагентное обучение с подкреплением (MARL)

2.1 От одного агента к нескольким

В первом модуле мы научились обучать агентов в одноагентной системе. Агент оставался в среде один: **он не сотрудничал и не взаимодействовал с другими агентами**.



Рис. 4: Коллаж сред, в которых вы обучали агентов с начала курса [оригинал]

При многоагентном обучении с подкреплением (MARL) в одной среде действует несколько агентов, **которые совместно используют эту среду и взаимодействуют друг с другом**.

Представьте, например, склад, на котором **нескольким роботам необходимо перемещаться, чтобы загружать и разгружать посылки**.



Рис. 5: Автономные складские роботы, иллюстрация upklyak (Freepik). [оригинал]

Или дорогу, по которой едут **несколько беспилотных автомобилей**.

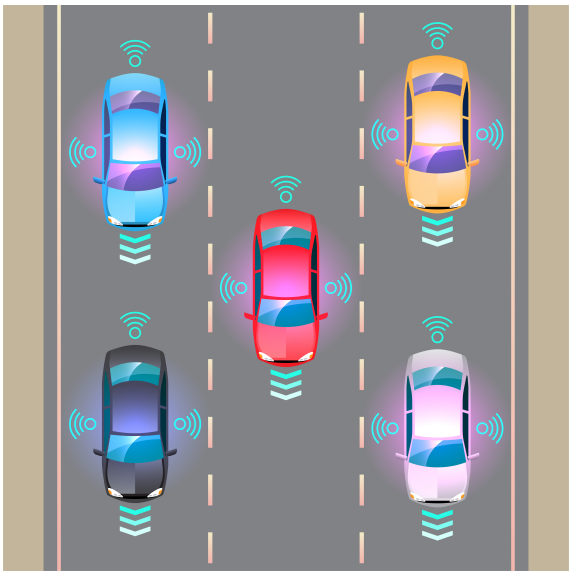


Рис. 6: Взаимодействие беспилотных автомобилей на дороге, иллюстрация jcomp (Freepik). [оригинал]

В этих примерах **несколько агентов взаимодействуют со средой и друг с другом**. Возникает необходимость описать многоагентную систему. Но сначала разберём типы многоагентных сред.

2.2 Типы многоагентных сред

Поскольку в многоагентной системе агенты взаимодействуют друг с другом, можно выделить несколько типов сред:

- *Кооперативные среды*: агенты стремятся **максимизировать общую выгоду**.

Например, на складе **роботы должны сотрудничать, чтобы загружать и разгружать посылки эффективно, то есть как можно быстрее**.

- *Конкурентные (состязательные) среды*: здесь каждый агент **стремится максимизировать собственную выгоду, одновременно снижая результат противника**.

Например, в теннисе **каждый агент стремится победить другого**.

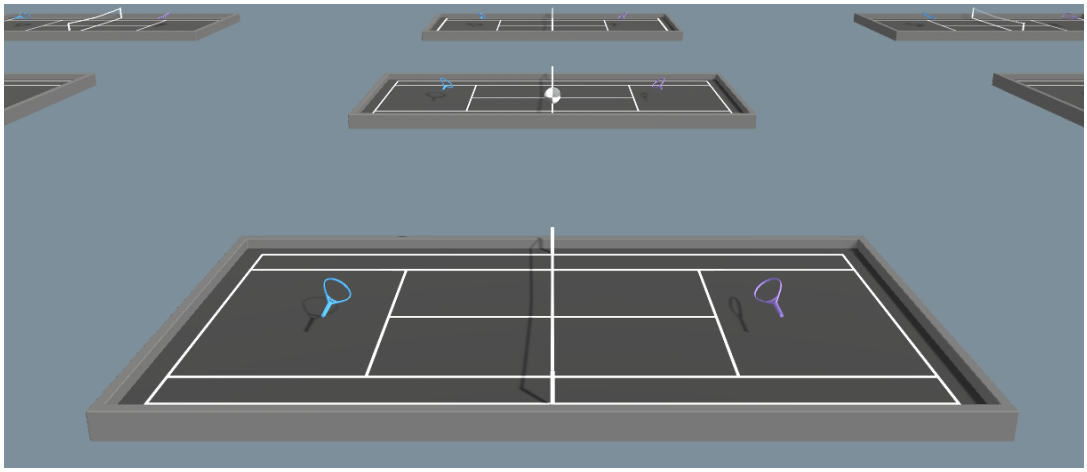


Рис. 7: Соперничество двух агентов в теннисе [оригинал]

- *Смешанные среды, сочетающие конкуренцию и сотрудничество*: как в SoccerTwos. Два агента входят в одну команду (синюю или фиолетовую); им нужно сотрудничать друг с другом и победить команду соперников.

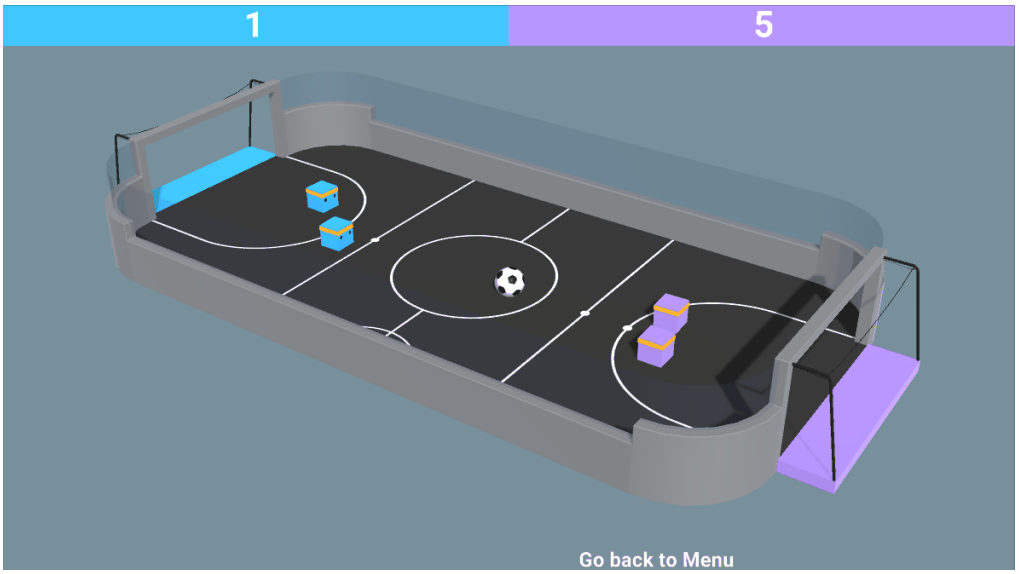


Рис. 8: SoccerTwos: футбольная среда Unity ML-Agents для игры два на два (команда Unity ML-Agents) [оригинальная анимация]

Теперь возникает вопрос: как проектировать такие многоагентные системы? Иначе говоря, **как обучать агентов в многоагентной постановке**?

3 Проектирование многоагентных систем

В этом разделе вы посмотрите замечательное введение в многоагентные системы от Брайана Дугласа.

Посмотреть оригинальную лекцию о проектировании многоагентных систем на YouTube

В этом видео Брайан объясняет принципы проектирования многоагентных систем. В качестве примера он рассматривает систему роботов-пылесосов и задаётся вопросом: **как организовать их сотрудничество**?

Есть два подхода к проектированию такой системы многоагентного обучения с подкреплением (MARL).

3.1 Децентрализованная система

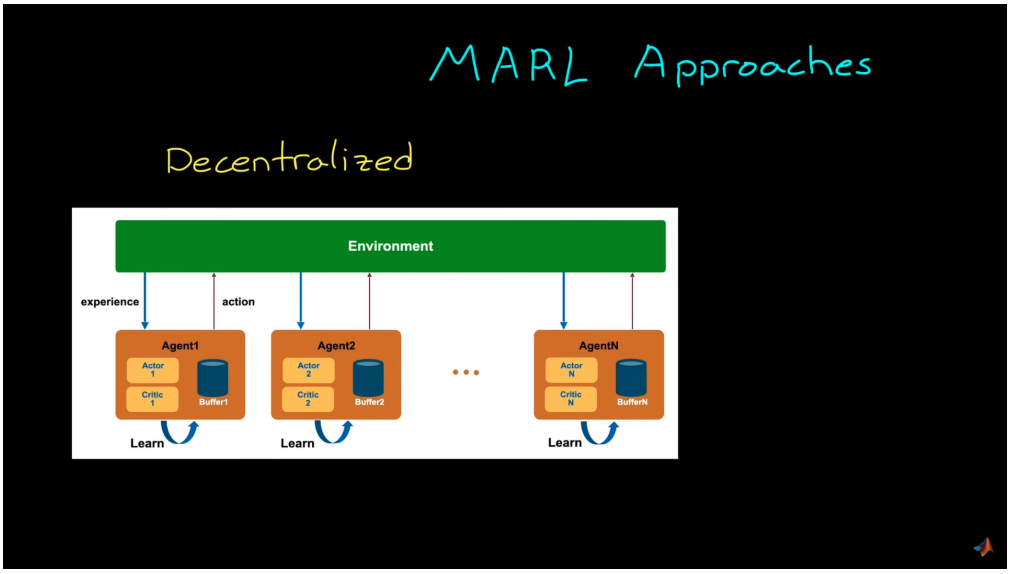


Рис. 9: Децентрализованное обучение нескольких роботов-пылесосов (Брайан Дуглас: введение в многоагентное RL) [оригинал]

При децентрализованном обучении **каждый агент обучается независимо от остальных**. В приведённом примере каждый пылесос учится убирать как можно большую площадь, **не учитывая действия других пылесосов (агентов)**.

Преимущество состоит в том, что **агенты не обмениваются информацией, поэтому таких пылесосов можно проектировать и обучать так же, как обычных одиночных агентов**.

Идея в том, что **обучаемый агент считает остальных агентов частью динамики окружающей среды**, а не самостоятельными агентами.

Однако главный недостаток такого подхода — **нестационарность среды**: базовый марковский процесс принятия решений со временем меняется, поскольку остальные агенты тоже взаимодействуют со средой. Для многих алгоритмов обучения с подкреплением это проблема: **в нестационарной среде они не способны гарантированно достичь глобального оптимума**.

3.2 Централизованный подход

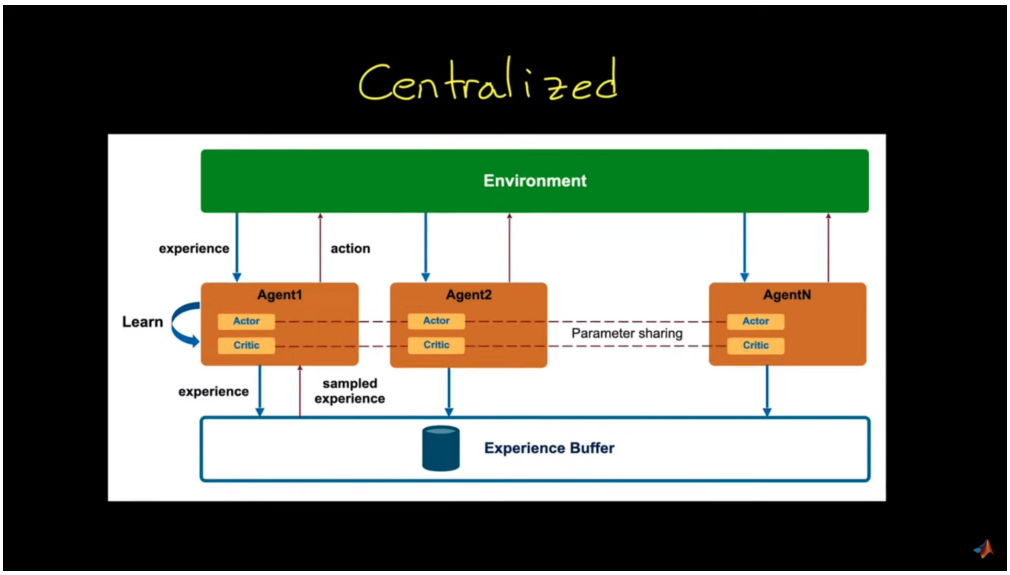


Рис. 10: Централизованное обучение нескольких агентов на основе общего опыта (Брайан Дуглас) [оригинал]

В этой архитектуре **существует процесс верхнего уровня, собирающий опыт всех агентов**, то есть буфер опыта. По собранным данным мы будем **обучать общую политику**.

Например, в задаче с роботами-пылесосами наблюдение включает:

- Карту покрытой пылесосами площади.
- Положения всех пылесосов.

Мы используем общий опыт, **чтобы обучить политику, управляющую всеми тремя роботами с наибольшей совокупной выгодой**. Каждый робот таким образом обучается на совместном опыте. Среда становится стационарной, поскольку все агенты рассматриваются как одна большая сущность и изменения их политик известны (ведь используется общая политика).

Подведём итог:

- При *децентрализованном подходе* мы **рассматриваем всех агентов независимо и не учитываем существование остальных**.
  - Каждый агент **считает других агентов частью окружающей среды**.
  - **Условия среды нестационарны**, поэтому гарантии сходимости отсутствуют.
- При *централизованном подходе*:
  - **Общая политика обучается на опыте всех агентов**.
  - Политика получает на вход текущее состояние среды и выдаёт совместные действия агентов.
  - Вознаграждение является общим.

4 Self-Play: классический метод обучения агентов в состязательных играх

Теперь, когда мы разобрали основы многоагентных систем, можно углубиться в тему. Как говорилось во введении, мы будем **обучать агентов в состязательной игре SoccerTwos — футболе 2 на 2**.

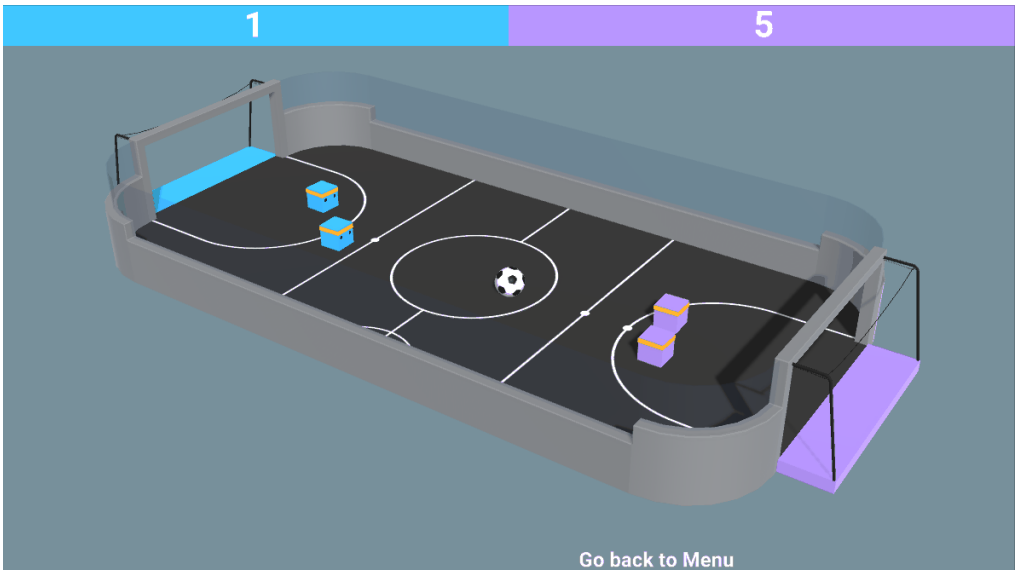


Рис. 11: SoccerTwos: футбольная среда Unity ML-Agents для игры два на два (команда Unity ML-Agents) [оригинальная анимация]

4.1 Что такое Self-Play?

Правильно обучить агентов в состязательной игре бывает **довольно сложно**.

С одной стороны, нужно найти достаточно хорошо обученного соперника, против которого будет играть наш агент. С другой — если соперник окажется слишком сильным, как наш агент сможет улучшать политику?

Представьте ребёнка, который только начал учиться играть в футбол. Игра против очень опытного футболиста будет бесполезной: ребёнку будет крайне трудно победить или хотя бы время от времени завладевать мячом. Он станет постоянно проигрывать, не успевая освоить хорошую стратегию.

Лучше всего было бы **подобрать соперника примерно того же уровня, что и агент, и повышать уровень соперника по мере улучшения самого агента**. Слишком сильный соперник не даст ничему научиться; слишком слабый заставит переобучиться на бесполезном поведении, которое не поможет против более сильного противника.

Этот подход называется *Self-Play (игра с самим собой)*. При Self-Play **агент использует в качестве соперника предыдущие копии самого себя, то есть своей политики**. Так он играет против сопоставимого по силе противника, сталкивается с посильным вызовом, постепенно улучшает политику и обновляет соперника, когда становится сильнее. Это способ последовательно усложнять противника и обеспечивать обучение по нарастающей.

Люди в соревнованиях учатся похожим образом:

- Сначала тренируются против соперника примерно своего уровня.
- Затем учатся на этом опыте и, приобретая навыки, переходят к более сильным соперникам.

То же самое происходит при Self-Play:

- Мы **начинаем с копии нашего агента в качестве соперника**, поэтому их уровни примерно одинаковы.
- Мы **учимся на игре с ним**, а по мере приобретения новых навыков **обновляем соперника, используя более свежую копию обучаемой политики**.

Сама идея Self-Play не нова. Её использовали ещё в системе игры в шашки Артура Сэмюэла в 1950-х годах и в TD-Gammon Джеральда Тезауро в 1995 году. Если хотите узнать больше об истории Self-Play, прочитайте отличную статью Эндрю Коэна.

4.2 Self-Play в ML-Agents

Self-Play встроен в библиотеку ML-Agents и управляется несколькими гиперпараметрами, которые мы рассмотрим. Однако основная задача, как подчёркивается в документации, — **найти баланс между уровнем навыка и способностью итоговой политики к обобщению, с одной стороны, и стабильностью обучения — с другой**.

Обучение против небольшого разнообразия медленно изменяющихся или неизменных соперников **делает обучение стабильнее. Но если соперники изменяются слишком медленно, возникает риск переобучения**.

Поэтому нужно контролировать следующие параметры:

- **Частоту смены соперников** с помощью `swap_steps` и `team_change`.
- **Число сохраняемых соперников** с помощью `window`. Чем больше значение `window`, тем разнообразнее будет пул соперников агента: в него войдут политики, полученные на более ранних этапах обучения.
- **Вероятность игры против текущей версии агента или соперника из пула** задаёт `play_against_latest_model_ratio`. Более высокое значение означает, что агент чаще играет против текущей версии соперника.
- **Число шагов обучения до сохранения нового соперника** задаёт `save_steps`. Большее значение позволяет получить соперников с более широким диапазоном навыков и, возможно, стилей игры, поскольку каждая версия политики обучается дольше.

Подробное описание этих гиперпараметров можно найти в соответствующем разделе документации.

4.3 Рейтинг Elo для оценки агента

Что такое рейтинг Elo?

В состязательных играх **суммарное вознаграждение не всегда является содержательной метрикой прогресса обучения**, поскольку оно **зависит также от уровня соперника**.

Вместо него применяется ***рейтинговая система Elo*** (названа по имени Арпада Эло), оценивающая **относительный уровень мастерства** двух участников из некоторой популяции в игре с нулевой суммой.

В игре с нулевой суммой один агент выигрывает, а другой проигрывает. Математически это ситуация, в которой выигрыш или потеря полезности одного участника **в точности уравновешивается выигрышем или потерей полезности остальных**. Поэтому суммарная полезность всех участников равна нулю.

Рейтинг Elo (часто начальное значение равно 1200) может сначала уменьшаться, но в ходе обучения должен постепенно расти.

Система Elo **опирается на поражения и ничьи в играх с другими участниками**. Следовательно, рейтинг игрока зависит **от рейтингов его соперников и результатов матчей с ними**.

Показатель Elo характеризует относительное мастерство игрока в игре с нулевой суммой. **Он называется относительным, поскольку зависит от результатов соперников**.

Основная идея состоит в том, что результаты игрока рассматриваются **как нормально распределённая случайная величина**.

Разница в рейтингах двух игроков служит **предиктором исхода матча**. Если игрок выигрывает, но вероятность его победы изначально была высокой, он получает совсем немного очков, поскольку уже считается значительно сильнее соперника.

После каждой игры:

- Победитель **получает очки за счёт проигравшего**.
- Число передаваемых очков определяется **разностью рейтингов двух игроков (отсюда относительность)**.
  - Если побеждает игрок с более высоким рейтингом →, он получает мало очков от игрока с низким рейтингом.
  - Если побеждает игрок с более низким рейтингом →, он получает много очков от игрока с высоким рейтингом.
  - При ничьей → игрок с более низким рейтингом получает немного очков от игрока с более высоким рейтингом.

Итак, если рейтинги игроков A и B равны R_A и R_B, то **ожидаемые результаты** вычисляются по формулам:

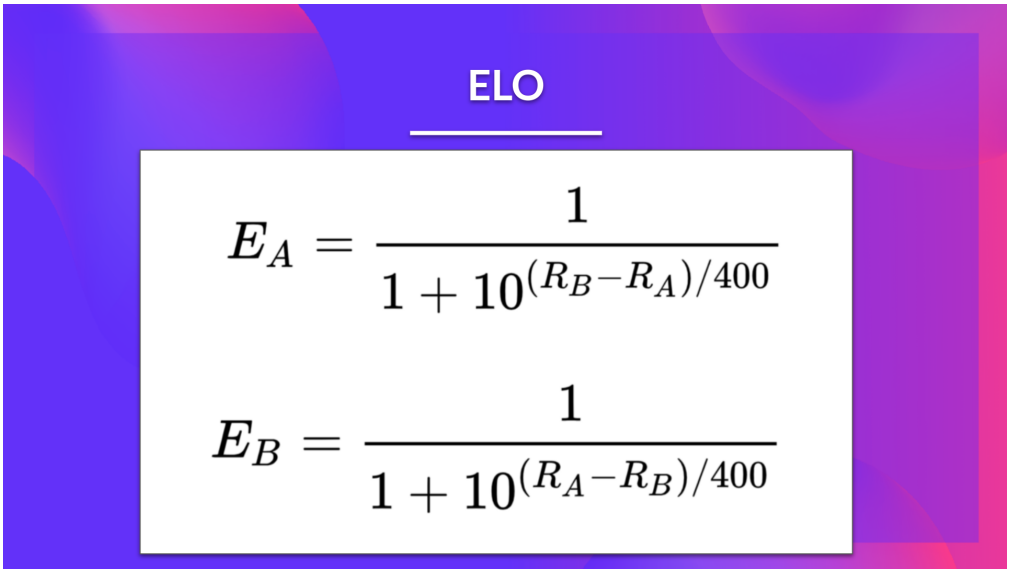


Рис. 12: Формула ожидаемого результата Elo для двух игроков [оригинал]

После окончания игры нужно обновить фактический рейтинг Elo каждого игрока. Применяется линейная корректировка, **пропорциональная величине, на которую результат превысил или не достиг ожидаемого**.

Также задаётся максимальное изменение рейтинга за игру — коэффициент K.

- $K = 16$ — для мастеров.
- $K = 32$ — для более слабых игроков.

Если игрок A имеет ожидаемый результат E_A, а фактический равен S_A, новый рейтинг рассчитывается по формуле:

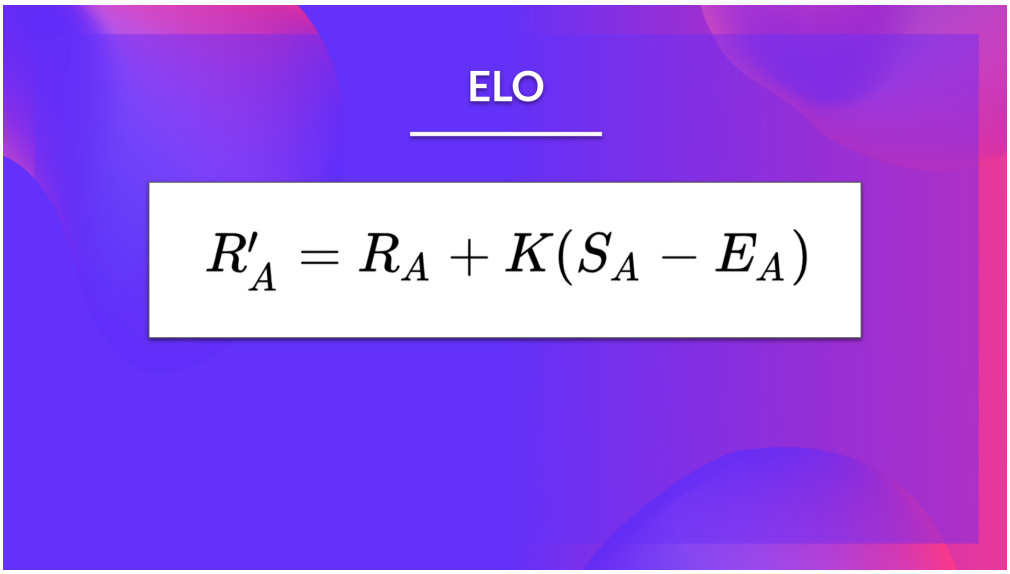


Рис. 13: Формула обновления рейтинга Elo после матча [оригинал]

Пример

Рассмотрим пример:

Рейтинг игрока A равен 2600.

Рейтинг игрока B равен 2300.

- Сначала вычислим ожидаемый результат:

$E_A = \frac{1}{1+10^{(2300-2600)/400}} = 0.849$

$E_B = \frac{1}{1+10^{(2600-2300)/400}} = 0.151$

- Если организаторы установили $K = 16$ и победил A, новый рейтинг составит:

$ELO_A = 2600 + 16 * (1 - 0.849) = 2602$

$ELO_B = 2300 + 16 * (0 - 0.151) = 2298$

- Если организаторы установили $K = 16$ и победил B, новый рейтинг составит:

$ELO_A = 2600 + 16 * (0 - 0.849) = 2586$

$ELO_B = 2300 + 16 * (1 - 0.151) = 2314$

Преимущества

Система Elo обладает несколькими преимуществами:

- Очки **всегда сбалансированы**: при неожиданном результате передаётся больше очков, но их общая сумма остаётся прежней.
- Это **самокорректирующаяся система**: победив слабого соперника, игрок получает лишь несколько очков.
- Система **подходит для командных игр**: для каждой команды вычисляется средний рейтинг, который затем используется в Elo.

Недостатки

- Elo **не учитывает индивидуальный вклад** каждого участника команды.
- Дефляция рейтинга: **чтобы поддерживать высокий рейтинг, необходимо долго сохранять высокий уровень игры**.
- **Нельзя напрямую сопоставлять исторические значения рейтинга**.

5 Практическое задание

Теперь, когда вы изучили основы многоагентных систем, пришло время обучить первых агентов в такой среде: **футбольную команду 2 на 2, которая должна победить соперников**.

Также вам предлагается участвовать в соревнованиях AI vs. AI, где обученный агент будет каждый день соревноваться с **агентами других участников и попадать в рейтинг**.

Для зачёта этого практического задания в рамках сертификации достаточно загрузить обученную модель. **Минимального порога результатов для зачёта нет**.

Подробнее о сертификации см. https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process.

Это практическое задание отличается от предыдущих: для получения хорошего результата **агентов потребуется обучать от четырёх до восьми часов**. Из-за риска ограничения времени в Colab авторы рекомендуют запускать обучение на собственном компьютере. Суперкомпьютер не нужен: обычного ноутбука для упражнения достаточно.

Приступим!

5.1 Что такое AI vs. AI?

AI vs. AI — инструмент с открытым исходным кодом, разработанный в Hugging Face для соревнования агентов, опубликованных на Hub, в многоагентных средах. Затем модели ранжируются в таблице лидеров.

Цель инструмента — обеспечить надёжную оценку: **сравнивая своего агента со множеством других, вы получаете хорошее представление о качестве его политики**.

Точнее, AI vs. AI состоит из трёх инструментов:

- *Система подбора соперников (matchmaking)* определяет пары моделей и запускает матчи фоновым заданием в Space.
- *Таблица лидеров* получает историю матчей и показывает рейтинги Elo моделей: https://huggingface.co/spaces/huggingface-projects/AIvsAI-SoccerTwos.
- *Демонстрационный Space* позволяет посмотреть, как агенты играют друг против друга: https://huggingface.co/spaces/unity/ML-Agents-SoccerTwos.

Кроме этих трёх инструментов участник курса cyllum создал SoccerTwos Challenge Analytics для просмотра подробных результатов матчей конкретной модели: https://huggingface.co/spaces/cyllum/soccertwos-analytics.

Подробности описаны в статье команды о системе AI vs. AI, но общая схема работы следующая:

- Каждые четыре часа алгоритм **загружает список всех доступных моделей для заданной среды (в нашем случае ML-Agents-SoccerTwos)**.
- Он формирует **очередь матчей с помощью алгоритма подбора соперников**.
- Матч моделируется в Unity без графического интерфейса, а **результат сохраняется** в Dataset: 1, если выиграла первая модель; 0,5 при ничьей; 0, если выиграла вторая.
- После завершения всех матчей очереди **обновляются рейтинги Elo всех моделей и таблица лидеров**.

Правила соревнования

Первое соревнование AI vs. AI **носит экспериментальный характер**: цель — улучшать инструмент благодаря отзывам участников. Поэтому во время соревнования **возможны сбои**. Однако **все результаты сохраняются в наборе данных, и вычисления можно перезапустить без потери информации**.

Чтобы модель корректно сравнивалась с остальными, необходимо соблюдать правила:

1. **Нельзя изменять пространство наблюдений или пространство действий агента**. Иначе модель не будет работать при оценивании.
2. **Пока нельзя применять собственный обучающий алгоритм**: необходимо использовать тренеры Unity ML-Agents.
3. Авторы предоставляют исполняемые файлы сред для обучения. При желании можно использовать редактор Unity, **но во избежание ошибок рекомендуется использовать готовые исполняемые файлы**.

Основное различие в результатах участников будут определять **выбранные гиперпараметры**.

Авторы постоянно улучшают учебные материалы. Поэтому **если вы нашли проблему в ноутбуке**, создайте обращение в GitHub-репозитории.

Общение с участниками и вопросы в Discord

- Для обмена советами и вопросами был создан канал `ai-vs-ai-challenge`.
- Если вы ещё не присоединились к Discord-серверу, перейдите по приглашению.

5.2 Шаг 0: установка ML-Agents и загрузка среды

Авторы советуют использовать conda как менеджер пакетов и создать новое окружение.

С помощью conda создадим окружение с именем `rl` и **Python 3.10.12**:

```
conda create --name rl python=3.10.12
conda activate rl
```

Для корректного обучения агентов и загрузки моделей на Hub нужно установить ML-Agents.

```
git clone https://github.com/Unity-Technologies/ml-agents
```

После клонирования репозитория (он занимает 2,63 ГБ) перейдём в него и установим пакеты.

```
cd ml-agents
pip install -e ./ml-agents-envs
pip install -e ./ml-agents
```

У пользователей Mac на Apple Silicon при установке возможны проблемы (например, не удаётся собрать пакет ONNX). Сначала стоит попробовать установить grpcio:

```
conda install grpcio
```

Также может помочь эта проблема на GitHub в официальном репозитории ML-Agents.

Наконец, нужно установить git-lfs: https://git-lfs.com/.

После установки необходимо добавить исполняемый файл обучающей среды. В зависимости от операционной системы скачайте соответствующий архив, распакуйте его и поместите в новую папку `training-envs-executables` внутри `ml-agents`.

В итоге исполняемый файл должен находиться по следующему пути:

`ml-agents/training-envs-executables/SoccerTwos`.

Windows: скачайте этот исполняемый файл.

Linux (Ubuntu): скачайте этот исполняемый файл.

Mac: скачайте этот исполняемый файл. Внимание: на Mac также необходимо выполнить `xattr -cr training-envs-executables/SoccerTwos/SoccerTwos.app`, чтобы запустить SoccerTwos.

5.3 Шаг 1: знакомство со средой

Среда называется `SoccerTwos`. Её разработала команда Unity ML-Agents. Документация доступна по ссылке.

Цель в этой среде — **забить мяч в ворота соперника и не дать ему попасть в собственные ворота**.

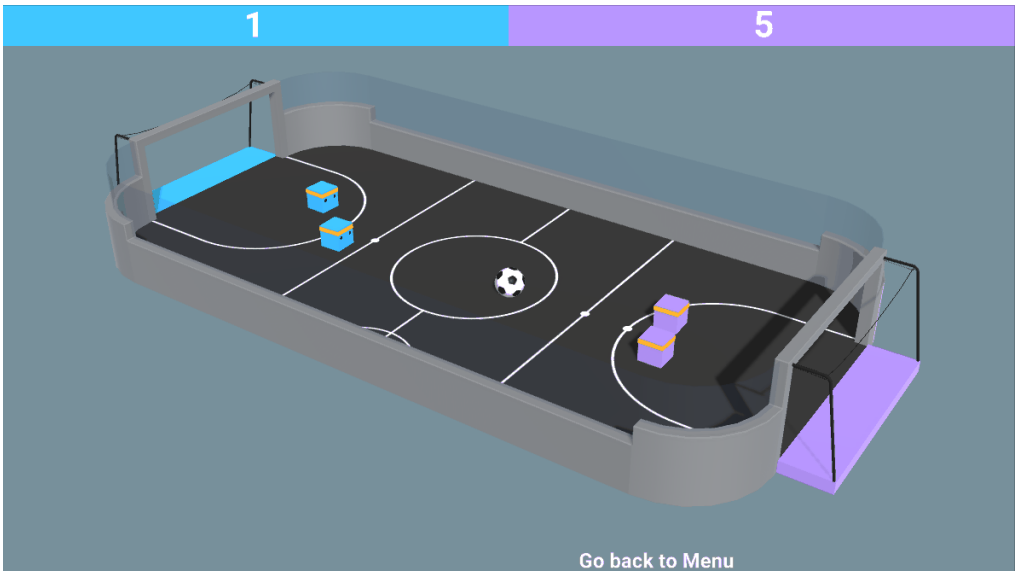


Рис. 14: SoccerTwos: футбольная среда Unity ML-Agents для игры два на два (команда Unity ML-Agents) [оригинальная анимация]

Функция вознаграждения

Функция вознаграждения имеет следующий вид:

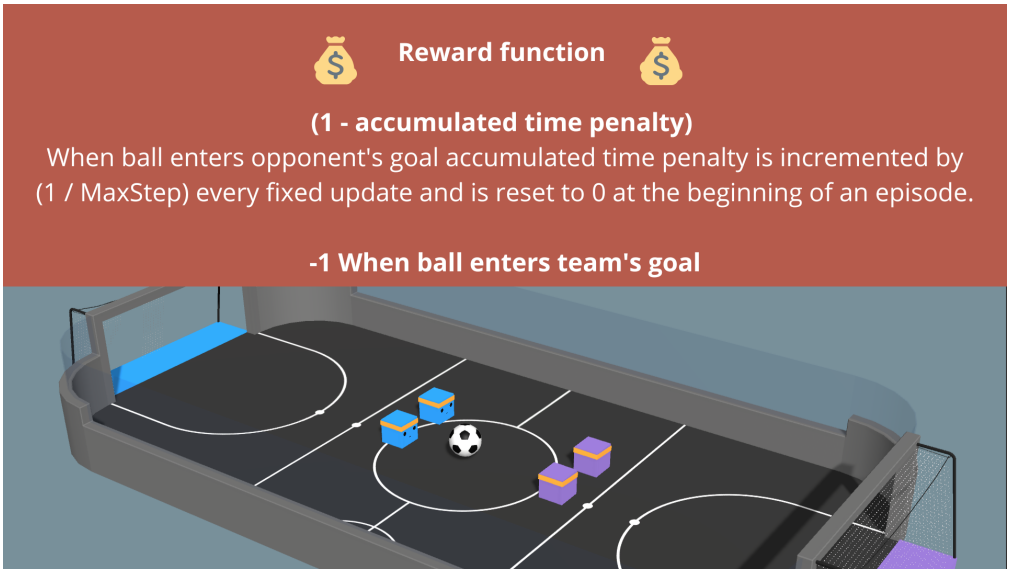


Рис. 15: SoccerTwos: функция вознаграждения среды [оригинал]

Пространство наблюдений

Пространство наблюдений состоит из векторов размерности 336:

- 11 лучей, направленных вперёд и распределённых по сектору 120 градусов (264 компоненты состояния).
- 3 луча, направленных назад и распределённых по сектору 90 градусов (72 компоненты состояния).

- Эти лучи способны распознавать шесть типов объектов:
  - Мяч.
  - Синие ворота.
  - Фиолетовые ворота.
  - Стена.
  - Синий агент.
  - Фиолетовый агент.

Пространство действий

Пространство действий состоит из трёх дискретных ветвей:



Рис. 16: SoccerTwos: дискретное и непрерывное пространства действий [оригинал]

5.4 Шаг 2: знакомство с MA-POCA

Мы уже знаем, как обучать агентов играть против других: **можно использовать Self-Play**. Для матчей 1 на 1 это прекрасный подход.

Но у нас матч 2 на 2, и в каждой команде по два агента. Как же **обучать коллективному поведению группы агентов**?

Как объясняется в блоге Unity, когда команда забивает гол, агенты обычно получают групповое вознаграждение (например, +1 минус штраф). Получается, что **каждый игрок команды получает вознаграждение, хотя вклад разных игроков в победу неодинаков**. Из-за этого трудно выяснить, как должен действовать каждый агент по отдельности.

Команда Unity ML-Agents разработала решение — новый многоагентный тренер *MA-POCA (Multi-Agent POsthumous Credit Assignment)*.

Идея проста, но эффективна: централизованный критик **обрабатывает состояния всех агентов команды и оценивает, насколько хорошо действует каждый из них**. Можно представить критика в роли тренера команды.

Благодаря этому каждый агент может **принимать решения только по собственным локальным наблюдениям** и **одновременно оценивать качество своего поведения в контексте всей группы**.

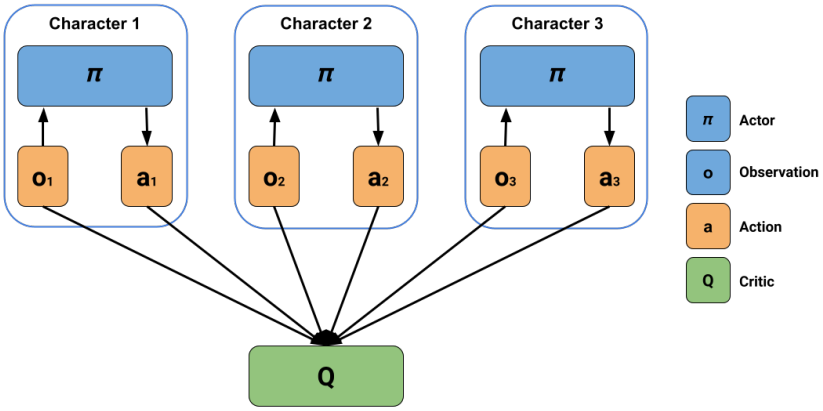


Рис. 17: MA-POCA: централизованное обучение и децентрализованное исполнение (источник: ML-Agents Plays Dodgeball) [оригинал]

Таким образом, решение — сочетать Self-Play с тренером MA-POCA (в конфигурации он называется `poca`). MA-POCA помогает обучать кооперативному поведению, а Self-Play — побеждать команду соперников.

Если хотите разобраться в MA-POCA глубже, прочитайте статью авторов и материалы из раздела дополнительной литературы.

5.5 Шаг 3: настройка файла конфигурации

Из модуля 5 мы знаем, что в ML-Agents **гиперпараметры обучения задаются в файлах `config.yaml`**.

Гиперпараметров довольно много. Чтобы разобраться в каждом, прочитайте пояснения в **документации**.

Используемый нами файл конфигурации расположен по пути `./config/poca/SoccerTwos.yaml`. Он выглядит так:

```
behaviors:
  SoccerTwos:
    trainer_type: poca
    hyperparameters:
      batch_size: 2048
      buffer_size: 20480
      learning_rate: 0.0003
      beta: 0.005
      epsilon: 0.2
      lambd: 0.95
      num_epoch: 3
      learning_rate_schedule: constant
    network_settings:
      normalize: false
      hidden_units: 512
      num_layers: 2
      vis_encode_type: simple
    reward_signals:
      extrinsic:
        gamma: 0.99
        strength: 1.0
    keep_checkpoints: 5
    max_steps: 5000000
    time_horizon: 1000
```

```
    summary_freq: 10000
    self_play:
      save_steps: 50000
      team_change: 200000
      swap_steps: 2000
      window: 10
      play_against_latest_model_ratio: 0.5
      initial_elo: 1200.0
```

По сравнению с Pyramids или SnowballTarget здесь появились дополнительные гиперпараметры Self-Play. Их настройка может иметь решающее значение для хорошего результата.

Совет: сопоставляйте смысл и рекомендуемые значения каждого параметра (особенно параметров Self-Play) с **документацией**.

После изменения файла конфигурации можно приступать к обучению агентов.

5.6 Шаг 4: запуск обучения

Для обучения агентов нужно **запустить `mlagents-learn` и указать исполняемый файл среды**.

Зададим четыре параметра:

1. `mlagents-learn <config>`: путь к файлу конфигурации гиперпараметров.
2. `-env`: путь к исполняемому файлу среды.
3. `-run_id`: идентификатор запуска обучения.
4. `-no-graphics`: отключение графического отображения в процессе обучения.

В зависимости от оборудования обучение на 5 миллионах временных шагов (рекомендуемое значение, но можно попробовать и 10 миллионов) займёт от пяти до восьми часов. В это время компьютером можно продолжать пользоваться, но лучше отключить переход в спящий режим, чтобы обучение не прервалось.

Команда запуска зависит от исполняемого файла для Windows, Ubuntu или macOS. Путь к файлу у вас может отличаться, поэтому проверьте его перед запуском.

Для Windows команда может выглядеть так:

```
mlagents-learn ./config/poca/SoccerTwos.yaml --env=./training-envs-executables/SoccerTwos.exe --run-
    id="SoccerTwos" --no-graphics
```

Для Mac команда может выглядеть так:

```
mlagents-learn ./config/poca/SoccerTwos.yaml --env=./training-envs-executables/SoccerTwos/
    SoccerTwos.app --run-id="SoccerTwos" --no-graphics
```

Исполняемый файл содержит восемь копий среды SoccerTwos.

Внимание: до двух миллионов шагов рейтинг Elo может почти не расти или даже опускаться ниже 1200. Это нормально: сначала агенты большую часть времени перемещаются по полю случайным образом, ещё не умея забивать голы.

Внимание: обучение можно остановить сочетанием Ctrl+C, но нажимать его следует только один раз. Перед завершением ML-Agents должен сформировать итоговый файл `.onnx`.

5.7 Шаг 5: публикация агента на Hugging Face Hub

Теперь, когда агенты обучены, **мы готовы загрузить их на Hub, чтобы участвовать в соревновании AI vs. AI и смотреть их игру в браузере**.

Чтобы поделиться моделью с сообществом, нужно выполнить ещё три шага:

1. Если у вас ещё нет аккаунта Hugging Face, зарегистрируйтесь: https://huggingface.co/join.
2. Войдите в аккаунт и получите токен аутентификации на сайте Hugging Face.

Создайте новый токен (https://huggingface.co/settings/tokens) **с правом записи (write)**.

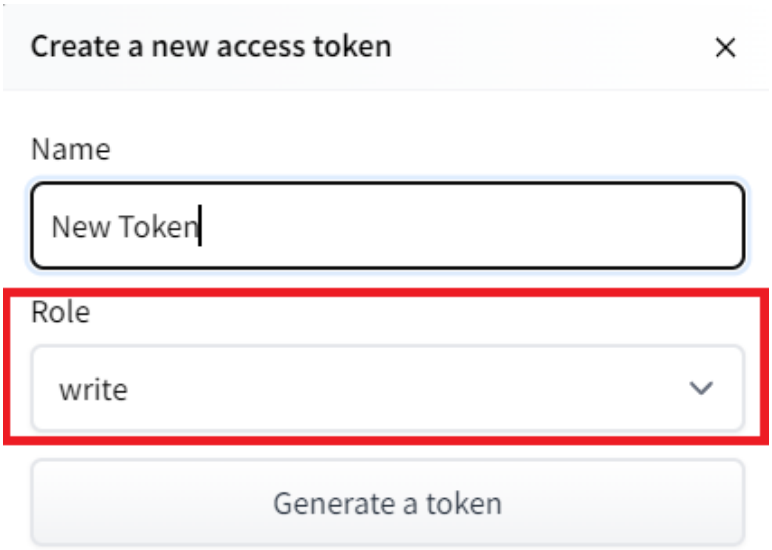


Рис. 18: Создание токена Hugging Face с правом записи [оригинал]

Скопируйте токен, запустите следующую команду и вставьте токен:

```
huggingface-cli login
```

Затем нужно выполнить `mlagents-push-to-hf`.

У команды четыре параметра:

1. `-run-id`: идентификатор запуска обучения.
2. `-local-dir`: каталог, в котором сохранён агент; обычно это `results/`, например `results/First Training`.
3. `-repo-id`: название создаваемого или обновляемого репозитория Hugging Face. Формат всегда `<ваш логин>/<имя репозитория>`. Если такого репозитория ещё нет, **он будет создан автоматически**.
4. `--commit-message`: сообщение коммита. Репозитории Hugging Face работают на основе Git, поэтому необходимо указать сообщение коммита.

В моём случае команда выглядит так:

```
mlagents-push-to-hf  --run-id="SoccerTwos" --local-dir="./results/SoccerTwos" --repo-id="
    ThomasSimonini/poca-SoccerTwos" --commit-message="First Push"`
```

```
mlagents-push-to-hf  --run-id= # Add your run id  --local-dir= # Your local dir  --repo-id= # Your
    repo id --commit-message="First Push"
```

Если всё прошло успешно, в конце появится примерно такое сообщение (но с другим адресом):

Модель опубликована на Hub. Посмотреть её можно здесь: https : / / huggingface . co / ThomasSimonini/poca-SoccerTwos.

Это ссылка на модель. В репозитории находятся карточка модели с инструкцией по использованию, данные TensorBoard и файл конфигурации. **Особенно удобно, что это Git-репозиторий:**

можно хранить разные коммиты, обновлять модель новыми публикациями и так далее.

5.8 Шаг 6: проверка готовности модели к AI vs. AI

После публикации на Hub **модель автоматически добавится в пул участников AI vs. AI**. До появления в таблице лидеров может пройти некоторое время, поскольку матчи проводятся каждые четыре часа.

Чтобы всё работало корректно, проверьте следующее:

1. У модели должен быть тег `ML-Agents-SoccerTwos`. По этому тегу выбираются модели для включения в соревнование. Откройте страницу модели и проверьте её теги.

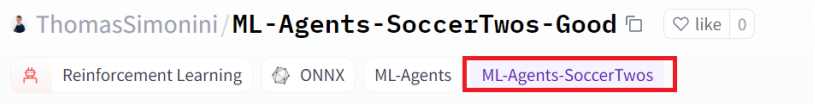


Рис. 19: Проверка тега ML-Agents-SoccerTwos у модели [оригинал]

Если тега нет, отредактируйте README и добавьте его.

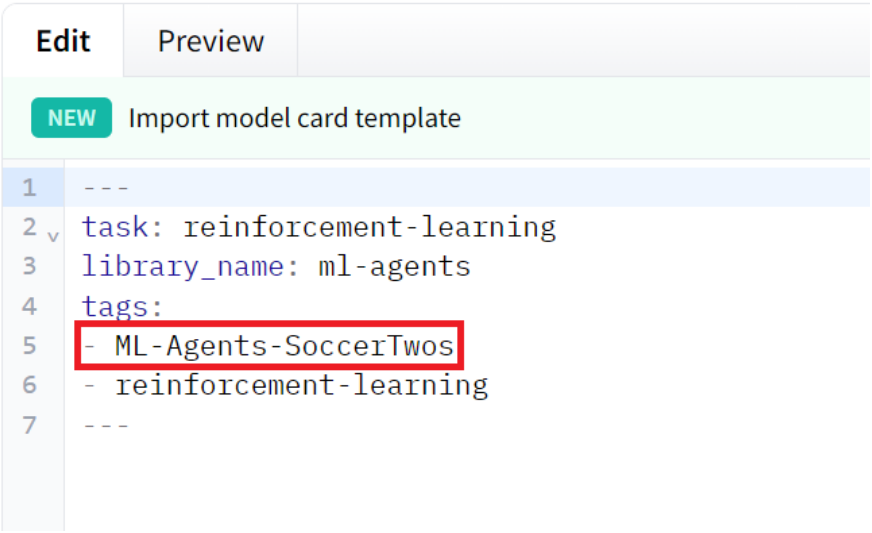


Рис. 20: Добавление тега SoccerTwos в карточку модели [оригинал]

2. У модели должен присутствовать файл `SoccerTwos.onnx`.

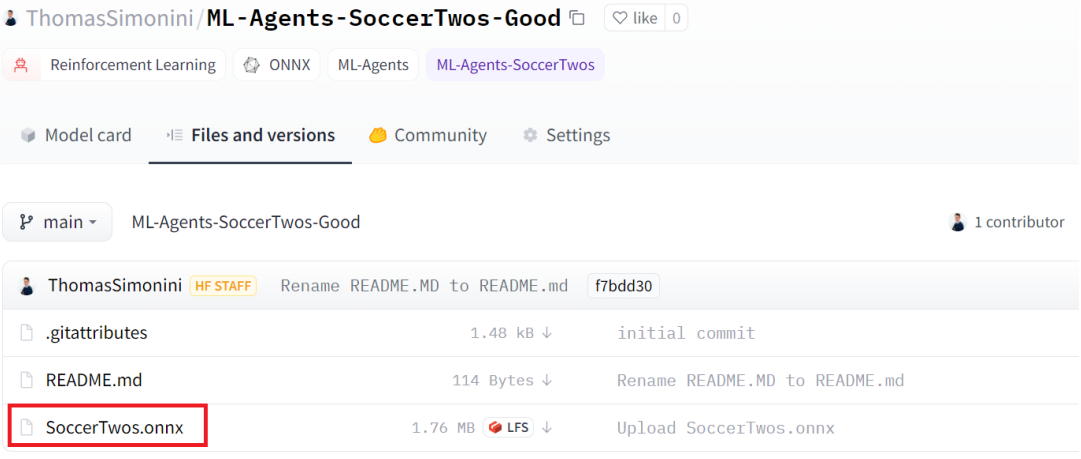


Рис. 21: Проверка загрузки файла SoccerTwos.onnx [оригинал]

Если вы собираетесь повторно обучать модель или создавать новую версию, авторы настоятельно рекомендуют публиковать её как новую модель на Hub.

5.9 Шаг 7: просмотр матчей в демонстрации

Теперь, когда модель участвует в AI vs. AI, **можно посмотреть, насколько хорошо она играет по сравнению с другими**: https://huggingface.co/spaces/unity/ML-Agents-SoccerTwos.

Для этого откройте демонстрацию:

- Выберите свою модель для синей команды (или фиолетовой, если хотите), а для второй команды — модель соперника. Лучше всего сравнивать свою модель с лидером рейтинга либо с базовой моделью.

Матчи, которые вы наблюдаете в реальном времени, не влияют на итоговую оценку, **но помогают увидеть качество игры агента**.

Не стесняйтесь делиться лучшими результатами своего агента в Discord, в канале `#rl-i-made-this`.

6 Проверка знаний

Лучший способ учиться и избегать иллюзии компетентности — **проверять себя**. Это позволит понять, **какие знания нужно закрепить**.

6.1 Q1: выберите правильное соответствие типов многоагентных сред

- Агенты стремятся максимизировать общую выгоду в _____ средах.
- Агенты стремятся максимизировать общую выгоду, одновременно уменьшая выгоду соперника, в _____ средах.
- **A.** конкурентных, кооперативных.
- **B.** кооперативных, конкурентных.

6.2 Q2: какие утверждения верны для децентрализованного обучения?

- **A.** Каждый агент обучается независимо от остальных.
- **B.** Данные о других агентах считаются просто данными окружающей среды.
- **C.** Если считать других агентов частью среды, среда становится стационарной.

6.3 Q3: какие утверждения верны для централизованного обучения?

- **A.** На основе взаимодействий всех агентов обучается одна общая политика.
- **B.** Вознаграждение является общим.
- **C.** При таком подходе среда стационарна.

6.4 Q4: своими словами объясните подход Self-Play

6.5 Q5: сопоставьте описания с гиперпараметрами Self-Play

- Вероятность игры против текущей версии агента по сравнению с противником из пула.
- Разнообразие (разброс) уровней подготовки возможных противников.
- Количество шагов обучения до создания нового противника.
- Частота смены противника.
- **A.** window, play_against_latest_model_ratio, save_steps, swap_steps+team_change
- **B.** play_against_latest_model_ratio, save_steps, window, swap_steps+team_change

- **C.** play_against_latest_model_ratio, window, save_steps, swap_steps+team_change
- **D.** swap_steps+team_change, save_steps, play_against_latest_model_ratio, window

6.6 Q6: каковы основные причины использования рейтинга Elo?

- **A.** Рейтинг учитывает разницу в уровне мастерства вашего агента и соперника.
- **B.** Хотя число передаваемых очков зависит от результата матча и уровней агентов, их общая сумма остаётся неизменной.
- **C.** Агенту легко долго сохранять высокий рейтинг.
- **D.** Рейтинг хорошо учитывает индивидуальный вклад каждого игрока в командный результат.

Поздравляем с завершением теста! Если какие-то темы оказались сложными, вернитесь к соответствующим разделам и закрепите знания.

6.7 Ответы и пояснения к тесту

Ответ на Q1 **Правильный ответ:** B

A. конкурентных, кооперативных. Пояснение: в кооперативной среде нужно максимизировать общую выгоду, а в конкурентной — также стремиться уменьшить результат соперника.

B. кооперативных, конкурентных.

Ответ на Q2 **Правильные ответы:** A, B

A. Каждый агент обучается независимо от остальных.

B. Данные о других агентах считаются просто данными окружающей среды.

C. Если считать других агентов частью среды, среда становится стационарной. Пояснение: при децентрализованном обучении агенты игнорируют других агентов как самостоятельные сущности и рассматривают их как часть среды. Но из-за этого среда постоянно изменяется и становится нестационарной.

Ответ на Q3 **Правильные ответы:** A, B, C

A. На основе взаимодействий всех агентов обучается одна общая политика.

B. Вознаграждение является общим.

C. При таком подходе среда стационарна.

Ответ на Q4 `Self-Play` — подход, при котором в роли противников используются копии агента с той же политикой, что и у обучаемого агента. Благодаря этому агент может учиться, соревнуясь с соперниками примерно своего уровня.

Ответ на Q5 **Правильный ответ:** C

A. window, play_against_latest_model_ratio, save_steps, swap_steps+team_change

B. play_against_latest_model_ratio, save_steps, window, swap_steps+team_change

C. play_against_latest_model_ratio, window, save_steps, swap_steps+team_change

D. swap_steps+team_change, save_steps, play_against_latest_model_ratio, window

Ответ на Q6 Правильные ответы: A, B

A. Рейтинг учитывает разницу в уровне мастерства вашего агента и соперника.

B. Хотя число передаваемых очков зависит от результата матча и уровней агентов, их общая сумма остаётся неизменной.

C. Агенту легко долго сохранять высокий рейтинг. Пояснение: здесь проявляется `дефляция рейтинга` (rating deflation) — для сохранения высокого рейтинга требуется долго демонстрировать высокий уровень игры.

D. Рейтинг хорошо учитывает индивидуальный вклад каждого игрока в командный результат. Пояснение: Elo учитывает результат всей команды, но не рассчитывает индивидуальный вклад её участников.

7 Заключение

На сегодня всё. Поздравляем с завершением модуля и практического задания!

Лучший способ учиться — практиковаться и экспериментировать. **Почему бы не обучить ещё одного агента с другой конфигурацией?**

Не забывайте время от времени проверять таблицу лидеров.

Увидимся в модуле 8!

7.1 Продолжайте учиться и оставайтесь молодцами

8 Дополнительная литература

8.1 Введение в многоагентные системы

- Обзор многоагентного обучения с подкреплением.
- Многоагентное обучение с подкреплением, Марк Ланкто.
- Пример многоагентной среды.
- Список различных многоагентных сред.
- Многоагентное обучение с подкреплением: независимые и сотрудничающие агенты.
- Работа с нестационарностью в глубоком многоагентном обучении с подкреплением.

8.2 Self-Play и MA-POCA

- Теория Self-Play и её реализация в ML-Agents.
- Обучение сложному кооперативному поведению с ML-Agents.
- Как агенты ML-Agents играют в вышибалы.
- О правильном и неправильном использовании поглощающих состояний в многоагентном обучении с подкреплением (MA-POCA).